

Python 06 Dictionaries

https://bioinfmsc8.bio.ed.ac.uk/AY25_BPSM16.html

Quick links:

Click on any [↑](#) to get back here!

- [Lists are not always the best](#)
- [What are paired data?](#)
- [Creating a dict \(dictionary\)](#)
- [Getting help for dictionaries](#)
- [Literal or constructor?](#)
- [Building up a dict](#)
- [Keys, values, items](#)
- [Using get](#)
- [Deleting, clearing...](#)
- [Counting dinucleotides with a dict](#)
- [Zero values](#)
- [Looping with dicts](#)
- [Looping over pairs](#)
- [Lookup vs. iteration](#)
- [Saving \(pickling\) our work](#)

Exercises

1. [Interrogating the user...](#)
2. [DNA translation](#)

-
- [important git/GitHub stuff at the end](#)

Dictionaries

Possibly useful other links (but very version dependent!)

[Dictionaries at python.org](#)

https://www.w3schools.com/python/python_dictionaries.asp

<https://realpython.com/python-dicts/>

📌 Introducing paired data

A simple Python programme....remember this?!

Count all the A bases in a DNA sequence

```
dna = "ATCGATCGATCGTACGCTGA"  
a_count = dna.upper().count("A")  
print(a_count)
```

5

That was pretty straightforward. How about this:

Count all four bases individually in a different DNA sequence

```
dna = "ATCGTATCGATGTACGCTGA"  
a_count = dna.upper().count("A")  
t_count = dna.upper().count("T")  
g_count = dna.upper().count("G")  
c_count = dna.upper().count("C")  
print(a_count, "t_count", "g_count", "c_count")
```

5, 6, 5, 4

Let's make this a bit more complicated, let's count all possible di-nucleotides: there are 16 possible combinations ($4 * 4$).

Count all possible dinucleotides in a DNA sequence

```
dna = "ATCGTATCGATGTACGCTGA"  
aa_count = dna.upper().count("AA")  
at_count = dna.upper().count("AT")  
ag_count = dna.upper().count("AG")
```

..... and so on

Let's make this a bit more complicated again, let's count all possible tri-nucleotides: there are 64 possible combinations ($4 * 4 * 4$).

Count all possible trinucleotides in a DNA sequence

```
dna = "ATCGTATCGATGTACGCTGA"  
aaa_count = dna.upper().count("AAA")  
aat_count = dna.upper().count("AAT")  
aag_count = dna.upper().count("AAG")
```

..... and so on

Count all possible 4-mers in a DNA sequence (256 possibles...)

```
dna = "ATCGTATCGATGTACGCTGA"  
aaaa_count = dna.upper().count("AAAA")  
aaat_count = dna.upper().count("AAAT")  
aaag_count = dna.upper().count("AAAG")
```

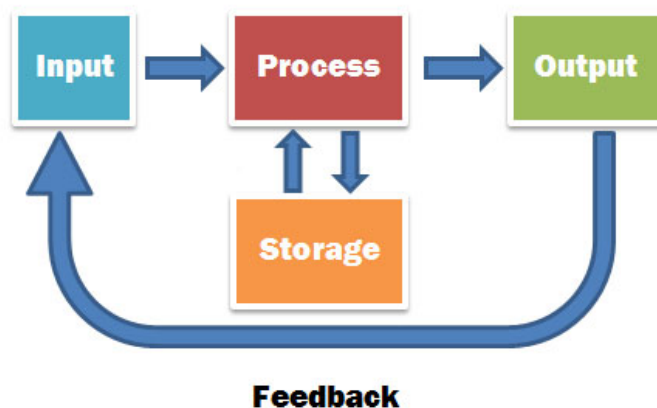
..... and so on

Count all possible 5-mers in a DNA sequence (1024 possibles...)

```
dna = "ATCGTATCGATGTACGCTGA"  
aaaaa_count = dna.upper().count("AAAAA")  
aaaat_count = dna.upper().count("AAAAT")  
aaaag_count = dna.upper().count("AAAAG")
```

..... and so on

How best to deal with this?



We could use a list to store the dinucleotide counts.

Programme/script to count dinucleotides in a sequence

Using a list to store all dinucleotides to be looked for in an input sequence

INPUT

```
dna = "ATCGTATCGATGTACGCTGA"
dinucleotides = ['AA', 'AT', 'AG', 'AC',
                  'TA', 'TT', 'TG', 'TC',
                  'GA', 'GT', 'GG', 'GC',
                  'CA', 'CT', 'CG', 'CC']
```

Set up a list to hold all the counts of each dinucleotide

```
all_counts = []
```

PROCESS with a `for` loop

```
for dinucleotide in dinucleotides:
    count = dna.upper().count(dinucleotide)
    all_counts.append(count)
```

OUTPUT

```
print(all_counts)
```

```
[0, 3, 0, 1, 2, 0, 2, 2, 2, 2, 0, 1, 0, 1, 3, 0]
```

Problem: there's no easy way to look up the count for a given dinucleotide. There's no longer any connection between the dinucleotides and the counts (other than in our minds!).

🏠 "Paired" data

The dinucleotides and their counts is an example of paired data, more correctly called key/value data pairs.

"Key" examples

dinucleotide

name

codon

sample

word

"Value" examples

count

protein sequence

amino acid residue

coordinates

definition

Python's data structure for storing this type of data is a dictionary, and is commonly called a **dict**.

📌 Creating dicts

The rules for making a `dict` are fairly simple:

1. they must start and end with curly brackets `{ }`
2. separate keys and values with colons `:`
3. separate each key/value pair (a dict "item") with a comma `,`

My favourite restriction enzymes for cutting DNA

```
enzymes = {  
'EcoRI' : 'GAATTC',  
'AvaII' : 'GGACC',  
'BisI'  : 'GCNGC'  
}
```

Can't recall what a restriction enzyme is? Check out [this link](#) or [this link](#)!

We often write dicts on multiple lines as they are easier to view.

Getting a single value is similar to a list, but instead of giving the index, we give the **key** for the value we want

"Index" doesnt work with square brackets!

```
enzymes[1]
```

```
Traceback (most recent call last):  
  File "<stdin>", line 1, in <module>  
KeyError: 1
```

"Index" doesnt work with curly brackets either!

```
enzymes{1}
```

```
File "<stdin>", line 1  
enzymes{1}
```

^
SyntaxError: invalid syntax

Incorrect way to retrieve the value using a key

enzymes[BisI]

Traceback (most recent call last):
 File "<stdin>", line 1, in <module>
NameError: name 'BisI' is not defined

Correct way to retrieve the value for a key from the dict

enzymes['BisI']

'GCNGC'

Retrieve the value for a key from the dict

and store it in variable

motif = enzymes['BisI']

print(motif)

GCNGC

```
# Fun things to do
```

```
# Our string variable dna
```

```
dna
```

```
'ATCGTATCGATGTACGCTGA'
```

```
# Our string variable: using list()
```

```
list(dna)
```

```
['A', 'T', 'C', 'G', 'T', 'A', 'T', 'C', 'G', 'A', 'T', 'G', 'T', 'A', 'C', 'G', 'C', 'T', 'G', 'A']
```

```
# Our string variable: using set()
```

```
set(dna)
```

```
{'C', 'A', 'G', 'T'}
```

but that isn't in alphabetical order All?

```
# Our string variable: using list(set())
```

```
list(set(dna))
```

```
['C', 'A', 'G', 'T']
```

```
# What type of object do we have?
```

```
type(dna)
```

```
<class 'str'>
```

```
type(set(dna))
```

```
<class 'set'>
```

```
type(list(set(dna)))
```

```
<class 'list'>
```

```
# How about our enzymes dictionary?
```

```
enzymes
```

```
{'EcoRI': 'GAATTC', 'Aval': 'GGACC', 'BisI': 'GCNGC'}
```

type(enzymes)

<class 'dict'>

set() gives us the keys

set(enzymes)

{'EcoRI', 'BisI', 'Avall'}

type(set(enzymes))

<class 'set'>

list(set(enzymes))

['EcoRI', 'BisI', 'Avall']

type(list(set(enzymes)))

<class 'list'>



This doesn't tell us about restriction enzymes,

it's about the `enzymes` object type! 😊

help(enzymes)

Help on dict object:

```
class dict(object)
| dict() -> new empty dictionary
| dict(mapping) -> new dictionary initialized from a mapping object's
|   (key, value) pairs
| dict(iterable) -> new dictionary initialized as if via:
|   d = {}
|   for k, v in iterable:
|       d[k] = v
| dict(**kwargs) -> new dictionary initialized with the name=value pairs
|   in the keyword argument list.  For example:  dict(one=1, two=2)
|
| Methods defined here:
|
| __contains__(self, key, /)
|     True if D has a key k, else False.
|
| __delitem__(self, key, /)
|     Delete self[key].
|
| __eq__(self, value, /)
|     Return self==value.
|
| __ge__(self, value, /)
|     Return self>=value.
|
| __getattr__(self, name, /)
|     Return getattr(self, name).
|
| __getitem__(...)
|     x.__getitem__(y) <==> x[y]
|
| __gt__(self, value, /)
|     Return self>value.
```

```
|  
|  __init__(self, /, *args, **kwargs)  
|      Initialize self.  See help(type(self)) for accurate signature.  
|  
|  __iter__(self, /)  
|      Implement iter(self).  
|  
|  __le__(self, value, /)  
|      Return self<=value.  
|  
|  __len__(self, /)  
|      Return len(self).  
|  
|  ...
```

Getting help using DOT TAB TAB

enzymes.

enzymes.clear(	enzymes.get(	enzymes.pop(	enzymes.update(
enzymes.copy(	enzymes.items(	enzymes.popitem(	enzymes.values(
enzymes.fromkeys(	enzymes.keys(	enzymes.setdefault(	

🏠 You might have noticed above that there was some code given like this:

dict()

This is because there are two forms/ways to make a dictionary in Python 3.x , *literal* or *constructor*. The examples we see today are done using the *literal* method with curly brackets. If you want to explore some of the differences, then have [a read of this](#), particularly towards the bottom of that page.

📌 Building up a dict

Just like we did for a list, we can create an empty `dict`, and add items to it one at a time.

```
# Create an empty dict called enzymes
```

```
enzymes = {}
```

```
# Add one key/value pair at a time
```

```
enzymes['EcoRI'] = 'GAATTC'  
enzymes['AvaII'] = 'GGACC'  
enzymes['BisI'] = 'GCNGC'  
enzymes['RanDomI'] = 'GCNYC'  
enzymes['RanDomI'] = 'NewYorkCITY'  
enzymes['RanDomI'] = enzymes['RanDomI'] + ' ABCDE,34'
```

```
# Show the list
```

```
print(enzymes)
```

```
{'EcoRI': 'GAATTC', 'AvaII': 'GGACC', 'BisI': 'GCNGC', 'RanDomI':  
'NewYorkCITY ABCDE,34'}
```

Make another similar dict, this time called `enzymes2`

```
enzymes2 = { 'EcoRI' : 'GAATTC', 'AvaI' : 'GGACC', 'BisI' : 'GCNGC' }
```

```
print(enzymes2)
```

```
{'EcoRI': 'GAATTC', 'AvaI': 'GGACC', 'BisI': 'GCNGC'}
```

```
print("\n".join(enzymes2))
```

```
EcoRI  
AvaI  
BisI
```

```
print("\n".join(enzymes2.keys()))
```

```
EcoRI  
AvaI  
BisI
```

```
print("\n".join(enzymes2.values()))
```

```
GAATTC  
GGACC  
GCNGC
```

How many elements are there in each dict?

```
len(enzymes)
```

```
4
```

```
len(enzymes2)
```

```
3
```



What are the `keys` in the enzymes dict?

```
enzymes.keys()
```

```
dict_keys(['EcoRI', 'AvaI', 'BisI', 'RanDomI'])
```

What are the `values` in the enzymes dict?

```
enzymes.values()
```

```
dict_values(['GAATTC', 'GGACC', 'GCNGC', 'NewYorkCITY ABCDE,34'])
```

# What are the	items	in the enzymes dict?
enzymes.items()		
dict_items([('EcoRI', 'GAATTC'), ('AvaI', 'GGACC'), ('BisI', 'GCNGC'), ('RanDomI', 'NewYorkCITY ABCDE,34')])		

Present or absent?
"RanDomI" in enzymes.keys()
True
"RanDomI" in enzymes2.keys()
False

```
# What keys are there? A different way...
```

```
list(enzymes)
```

```
['EcoRI', 'Avall', 'Bisl', 'RanDoml']
```

```
# For fun only, let's sort the keys alphabetically
```

```
list(enzymes).sort()
```

```
# That didnt work!
```

```
list(enzymes)
```

```
['EcoRI', 'Avall', 'Bisl', 'RanDoml']
```

```
# Try again...!
```

```
foo = list(enzymes).sort()
```

```
foo
```

```
# Why no output?
```

```
type(foo)
```

```
< class 'NoneType' >
```

```
# Try again...!
```

```
foo = list(enzymes)
```

```
foo
```

```
['EcoRI', 'Avall', 'Bisl', 'RanDoml']
```

```
foo.sort()
```

Correctly sorted output this time...!
foo
['Avall', 'Bisl', 'EcoRI', 'RanDoml']



Retrieve a value using a specific key

```
print(enzymes['EcoRI'])
```

```
GAATTC
```

Retrieve a value using a specific key using get()

Note the use of () when using get

```
enzymes.get('EcoRI')
```

```
'GAATTC'
```

```
enzymes.get('RanDomI')
```

```
'NewYorkCITY ABCDE,34'
```

If it isn't there, you get no output

```
enzymes.get('novel')
```

If not using get(), if it isn't there, Python gives an error...

```
enzymes['novel']
```

```
Traceback (most recent call last):  
  File "<stdin>", line 1, in <module>  
KeyError: 'novel'
```

Add the 'novel' key/value pair, this time our value is an integer

```
enzymes['novel'] = 3
```

Can we evaluate the enzymes.keys() component?

```
"novel" in enzymes.keys()
```

```
True
```

Can we evaluate the `enzymes.values()` component?

"novel" in `enzymes.values()`

False

3 in `enzymes.values()`

True

In the above values examples, **we DON'T know** what their keys are, just that the value is or isn't there....

```
# Our current contents of dictionary enzymes
```

```
enzymes
```

```
{'EcoRI': 'GAATTC', 'Avall': 'GGACC', 'BisI': 'GCNGC', 'RanDomI':  
'NewYorkCITY ABCDE,34', 'novel': 3}
```

```
# Add another key/value pair, this time our key looks like an integer
```

```
enzymes['5'] = 'GATTTACTC'
```

```
enzymes
```

```
{'EcoRI': 'GAATTC', 'Avall': 'GGACC', 'BisI': 'GCNGC', 'RanDomI':  
'NewYorkCITY ABCDE,34', 'novel': 3, '5': 'GATTTACTC'}
```

```
# Add another key/value pair, this time our key is an integer
```

```
enzymes[5] = 'GATTT'
```

```
enzymes
```

```
{'EcoRI': 'GAATTC', 'Avall': 'GGACC', 'BisI': 'GCNGC', 'RanDomI':  
'NewYorkCITY ABCDE,34', 'novel': 3, '5': 'GATTTACTC', 5: 'GATTT'}
```

```
# Add another key/value pair, an integer again
```

```
enzymes[1] = 'GATTTGGG'
```

```
enzymes
```

```
{'EcoRI': 'GAATTC', 'Avall': 'GGACC', 'BisI': 'GCNGC', 'RanDomI':  
'NewYorkCITY ABCDE,34', 'novel': 3, '5': 'GATTTACTC', 5: 'GATTT', 1:  
'GATTTGGG'}
```

The thing that goes inside the square brackets is **always the key**, irrespective of whether we are setting a value or retrieving a value, and irrespective of whether it is a string, an integer, or even a floating point number!

```
yes_i_CAN_do_this = { 1.4: 'a floating point key, and a string value' }
```

yes_i_CAN_do_this

{1.4: 'a floating point key, and a string value'}

yes_i_CAN_do_this[1.4]

'a floating point key, and a string value'

Can we use indexes on the enzymes dict: is this actually an index?

```
enzymes[1]
```

```
'GATTTGGG'
```

```
enzymes
```

```
{'EcoRI': 'GAATTC', 'AvaI': 'GGACC', 'BisI': 'GCNGC', 'RanDomI':  
'NewYorkCITY ABCDE,34', 'novel': 3, '5': 'GATTTACTC', 5: 'GATTT', 1:  
'GATTTGGG'}
```

Can we use indexes on the enzymes dict: is this actually an index?

```
enzymes[2]
```

```
Traceback (most recent call last):  
  File "<stdin>", line 1, in <module>  
KeyError: 2
```

Can we use indexes on enzymes.items?

```
enzymes.items[2]
```

```
Traceback (most recent call last):  
  File "<stdin>", line 1, in <module>  
TypeError: 'builtin_function_or_method' object is not subscriptable
```

Can we use indexes on the enzymes.items?

```
enzymes.items()
```

```
dict_items([('EcoRI', 'GAATTC'), ('AvaI', 'GGACC'), ('BisI', 'GCNGC'), ('RanDomI', 'NewYo
```

```
enzymes.items()[2]
```

```
Traceback (most recent call last):  
  File "<stdin>", line 1, in <module>  
TypeError: 'dict_items' object does not support indexing
```

Can we use indexes on the enzymes.keys? Yes, via a list...

```
list(enzymes.keys())
```

```
['EcoRI', 'AvaI', 'BisI', 'RanDomI', 'novel', '5', 5, 1]
```

```
list(enzymes.keys())[2]
```

```
'BisI'
```

```
# Can we use indexes on the enzymes.values? Yes, via a list...
```

```
list(enzymes.values())
```

```
['GAATTC', 'GGACC', 'GCNGC', 'NewYorkCITY ABCDE,34', 3, 'GATTTACTC',  
'GATT', 'GATTGGG']
```

```
list(enzymes.values())[2]
```

```
'GCNGC'
```

```
# Can we use indexes on the enzymes.items? Yes, via a list...
```

```
list(enzymes.items())[2]
```

```
('BisI', 'GCNGC')
```

```
list(enzymes.items())[2:]
```

```
[('BisI', 'GCNGC'), ('RanDomI', 'NewYorkCITY ABCDE,34'), ('novel', 3), ('5', 'GATTTACTC'),
```

```
list(enzymes.items())[-1]
```

```
(1, 'GATTGGG')
```

OUR DICTIONARIES ARE ORDERED BY INSERTION ORDER

From Python 3.6 onwards, the standard dict type maintains insertion order by default.

Five years ago, we were using Python 3.4.2, which had a "random" order that wasn't (obviously) determined by what we did... most disconcerting!

In 2020, to maintain the order in which we inserted things (if that was what we wanted), we needed to use an ordered dictionary.

```
# The OLD way, using an ordered dictionary...
```

```
from collections import OrderedDict
```

```
enzymes=OrderedDict()
```

Make sure you know what version of Python 3 you are running!

```
# I am on msc8
```

```
import sys
```

```
print(sys.version)
```

```
'3.12.3 (main, Aug 14 2025, 17:47:21) [GCC 13.3.0]'
```

```
print(sys.version_info)
```

```
sys.version_info(major=3, minor=12, micro=3, releaselevel='final', serial=0)
```

```
list(sys.version_info)
```

```
[3, 12, 3, 'final', 0]
```



Delete the "RanDomI" entry

```
enzymes['RanDomI']
```

```
'NewYorkCITY ABCDE,34'
```

```
del enzymes['RanDomI']
```

Has the "RanDomI" entry gone?

```
enzymes['RanDomI']
```

```
Traceback (most recent call last):  
  File "<stdin>", line 1, in <module>  
KeyError: 'RanDomI'
```

What is left?

```
enzymes
```

```
{'EcoRI': 'GAATTC', 'AvaI': 'GGACC', 'BisI': 'GCNGC', 'novel': 3, '5':  
'GATTACTC', 5: 'GATTT', 1: 'GATTGTTGGG'}
```

Remove all entries

```
enzymes.clear()
```

What is left?

```
enzymes
```

```
{}
```

Delete entire dictionary

```
del enzymes
```

What is left?

```
enzymes
```

```
Traceback (most recent call last):  
  File "<stdin>", line 1, in <module>
```

[Give feedback](#)
[Ask for help](#)

NameError: name 'enzymes' is not defined

📌 Counting dinucleotides with a dict

How does all this help us with our dinucleotides problem?! Here's how we store the counts in a `dict`. We start with an empty dict, and add one key/value pair for each dinucleotide:

```
# Programme/script to count dinucleotides in a sequence
```

```
# Using a list to store all dinucleotides to be looked for in a new input sequence
```

```
# INPUT
```

```
dna = "AATGATGAACGAC"
dinucleotides=[]
for first in 'ATGC' :
    for second in 'ATGC' :
        dinucleotides.append(str(first)+str(second))
```

```
dinucleotides
```

```
['AA', 'AT', 'AG', 'AC', 'TA', 'TT', 'TG', 'TC', 'GA', 'GT', 'GG', 'GC', 'CA', 'CT', 'CG', 'CC']
```

```
# Set up an empty dict to hold all the counts of each dinucleotide
```

```
all_counts = {}
```

```
# PROCESS
```

```
# Loop through each element of the dinucleotides list and
```

```
# put the counts into the dict as a new key/value pair
```

```
for dinucleotide in dinucleotides:
    count = dna.upper().count(dinucleotide)
    print("count is " + str(count) + " for " + dinucleotide)
    all_counts[dinucleotide] = count
```

```
# Output is:
```

```
count is 2 for AA
count is 2 for AT
count is 0 for AG
count is 2 for AC
count is 0 for TA
count is 0 for TT
count is 2 for TG
count is 0 for TC
count is 3 for GA
```

```
count is 0 for GT
count is 0 for GG
count is 0 for GC
count is 0 for CA
count is 0 for CT
count is 1 for CG
count is 0 for CC
```

What key/value pairs does the all_counts dict hold now?

```
print(all_counts)
```

```
{'AA': 2, 'AT': 2, 'AG': 0, 'AC': 2, 'TA': 0, 'TT': 0, 'TG': 2, 'TC': 0, 'GA': 3, 'GT': 0, 'GG': 0, 'GC': 0, 'CA': 0, 'CT': 0, 'CG': 1, 'CC': 0}
```

How many GA dinucleotides were there?

Use the key to get the value

```
all_counts['GA']
```

```
3
```

How many TGA "dinucleotides" were there?

```
all_counts["TGA"]
```

```
Traceback (most recent call last):
  File "<stdin>", line 1, in <module>
KeyError: 'TGA'
```

Using get() ?

```
all_counts.get("TGA")
```

Out of interest, how many TGA TRI nucleotides were there?

```
dna
```

```
'AATGATGAACGAC'
```

```
dna.upper().count("TGA")
```

```
2
```

📌 Removing zero counts

Many of the counts may be zero (and for bigger kmers, nearly all the counts will be zero), so if we wanted to remove them, how might we do it?

Solution: when counting each dinucleotide, we just store the counts that are greater than zero: the `dict` will be smaller in size, and thus might be quicker for Python to search/manipulate

Programme/script to count dinucleotides in a sequence

INPUT

```
dna = "AATGATCGATCGTACGCTGA"
nonzero_counts = {}
dinucleotides = ['AA', 'AT', 'AG', 'AC',
                  'TA', 'TT', 'TG', 'TC',
                  'GA', 'GT', 'GG', 'GC',
                  'CA', 'CT', 'CG', 'CC']
```

PROCESS

Loop through each element of the dinucleotides list and

put the counts into the `dict` as a new key/value pair

IF there are any

```
for dinucleotide in dinucleotides:
    howmany = dna.upper().count(dinucleotide)
    if howmany > 0:
        nonzero_counts[dinucleotide] = howmany
```

Show contents of dict

```
print(nonzero_counts)
```

```
{'AA': 1, 'AT': 3, 'AC': 1, 'TA': 1, 'TG': 2, 'TC': 2, 'GA': 3, 'GT': 1, 'GC': 1, 'CT': 1, 'CG': 3}
```

Retrieve some counts using their keys

```
nonzero_counts['GA']
```

```
3
```

There were no AG dinucleotides, let's check

nonzero_counts['AG']

```
Traceback (most recent call last):  
  File "<stdin>", line 1, in <module>  
KeyError: 'AG'
```

The get() method lets us specify a default for when the key isn't found

nonzero_counts.get('GA',0)

3

We had already noticed that there were no AG dinucleotides, let's check again

nonzero_counts.get('AG',0)

0

📌 Looping with dicts

The `keys()` method returns a `list` of all the keys in a dict.

```
all_counts.keys()
```

```
dict_keys(['AA', 'AT', 'AG', 'AC', 'TA', 'TT', 'TG', 'TC', 'GA', 'GT', 'GG', 'GC',  
'CA', 'CT', 'CG', 'CC'])
```

so we can easily answer questions that require us to look at all the existing key/value pairs.

```
# Which dinucleotides occur exactly twice in the sequence?
```

```
for dinucleotide in all_counts.keys():  
    howmany = all_counts.get(dinucleotide)  
    if howmany == 2:  
        print(dinucleotide)
```

```
AA  
AT  
AC  
TG
```

📌 Looping over pairs

This pattern of iterating over keys and looking up values is common, so it's useful to know about another feature of dicts, namely the `items` method.

```
# Here is the all_counts dict as made earlier
```

```
all_counts
```

```
{'AA': 2, 'AT': 2, 'AG': 0, 'AC': 2, 'TA': 0, 'TT': 0, 'TG': 2, 'TC': 0, 'GA': 3, 'GT': 0, 'GG': 0, 'GC': 0, 'CA': 0, 'CT': 0, 'CG': 1, 'CC': 0}
```

```
# Here are the all_counts keys, which is a list
```

```
all_counts.keys()
```

```
dict_keys(['AA', 'AT', 'AG', 'AC', 'TA', 'TT', 'TG', 'TC', 'GA', 'GT', 'GG', 'GC', 'CA', 'CT', 'CG', 'CC'])
```

```
# Here are the all_counts values, which is also a list
```

```
all_counts.values()
```

```
dict_values([2, 2, 0, 2, 0, 0, 2, 0, 3, 0, 0, 0, 0, 0, 1, 0])
```

```
# Here are the all_counts items (i.e. key/value pairs), which is also a list
```

```
all_counts.items()
```

```
dict_items([('AA', 2), ('AT', 2), ('AG', 0), ('AC', 2), ('TA', 0), ('TT', 0), ('TG', 2), ('TC', 0), ('GA', 3), ('GT', 0), ('GG', 0), ('GC', 0), ('CA', 0), ('CT', 0), ('CG', 1), ('CC', 0)])
```

We are able to use a "shortcut" `for` loop to go through the keys and their values of the dict

```
# Say we want to loop through the all_counts dict to find those with 2 counts
```

In this loop, we decide that the variable names for

the keys will be `dinuc`

and values will be `howmany`

```
for dinuc, howmany in all_counts.items():  
    if howmany == 2:  
        print(dinuc)
```

AA

AT

AC

TG

📌Lookup vs. iteration

Remember, we don't *need* to write a loop if we just want to get a single value!

If we are looking for the count for 'AT', we could do this:

The long way

```
for dinuc, howmany in all_counts.items():  
    if dinuc == 'AT':  
        print(howmany)
```

or we could just ask for the value directly using:

The short way

```
all_counts.get('AT')
```

```
2
```

📌 Pickling an object

Up until now, we haven't "saved" anything we have done in Python, other than our scripts (but they were saved as plain text using `nano` or `vi` or whatever...).

Sometimes, we might want to save things (objects) we have generated in our Python "bubble", rather than regenerate them each time. To do this, we do something called "pickle an object" ("pickle" refers to a historical process of preserving edible products in an acid solution, usually vinegar, or in salt solution (brine)). In Python3, this method can be used for many different kinds of objects, including dictionaries.

Pickling involves "saving" things as serialized objects in a byte stream binary format as a file on our system.

Let's do it for the `all_counts` dictionary.

```
all_counts
```

```
{'AA': 2, 'AT': 2, 'AG': 0, 'AC': 2, 'TA': 0, 'TT': 0, 'TG': 2, 'TC': 0, 'GA': 3, 'GT': 0,
```

```
# Import the module we need
```

```
import pickle
```

```
# Choose a file name, open a file connection, make it writable and binary
```

```
coolfilename="pickled_all_counts"  
pcon = open(coolfilename, "wb")
```

```
# Dump our dictionary all_counts through the file connection
```

```
pickle.dump(all_counts, pcon)
```

```
# Close the file connection
```

```
pcon.close()
```

```
# Check things out!
```

```
os.system("ls -l")
```

```
pickled_all_counts
__pycache__
0
```

What's that `__pycache__`?

```
os.system("wc pickled_all_counts")
```

```
0 1 128 pickled_all_counts
0
```

```
os.system("cat pickled_all_counts")
```

```
u) (AA K AT K AG K AC K TA K TT K TG K TC K GA K GT K GG K GC K CA K CT K CG K CC Ku.0
```

"Loading" a pickled object back into a python session involves essentially the same steps as reading in a plain text file: make a connection, read contents, close the connection.

```
my_new_counts = pickle.load(open(coolfilename,"rb"))
```

```
my_new_counts
```

```
{'AA': 2, 'AT': 2, 'AG': 0, 'AC': 2, 'TA': 0, 'TT': 0, 'TG': 2, 'TC': 0, 'GA': 3, 'GT': 0,
```

```
# Are they the same?
```

```
all_counts == my_new_counts
```

```
True
```

For more details on pickling, this is a [good overview from DataCamp](#).

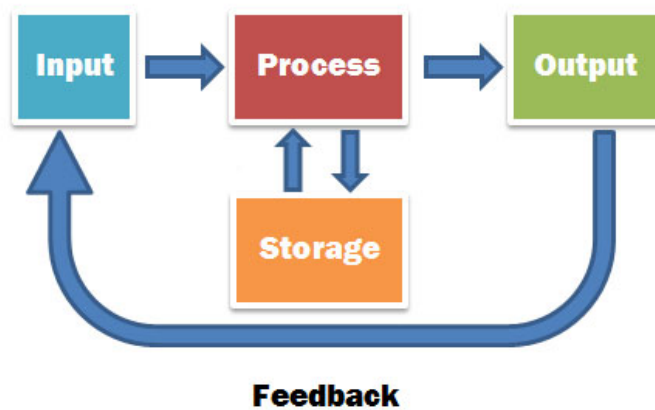
Exercises

Let's try and be tidy...

Make a directory within your homespace for today's work, and change directory to it.

```
mkdir ~/Exercises/Lecture16
```

```
cd ~/Exercises/Lecture16
```



📌Exercise #1: Interrogating the user...

With the aid of a `function` and a `dictionary`, write an `interactive` Python programme/script that will ask the user the following questions

1. What's your name?
2. How old are you?
3. What is your favourite colour?
4. Do you like Python?
5. The world is flat: True or False?

and then, based on their answers, make some comments back to them (this doesn't have to be at all serious, it's the methods used that we are trying out here....!)

📌Exercise #2:DNA translation

Here's a dict that stores a codon usage table for translation:

```
gencode = {
'ATA': 'I', 'ATC': 'I', 'ATT': 'I', 'ATG': 'M',
'ACA': 'T', 'ACC': 'T', 'ACG': 'T', 'ACT': 'T',
'AAC': 'N', 'AAT': 'N', 'AAA': 'K', 'AAG': 'K',
'AGC': 'S', 'AGT': 'S', 'AGA': 'R', 'AGG': 'R',
'CTA': 'L', 'CTC': 'L', 'CTG': 'L', 'CTT': 'L',
'CCA': 'P', 'CCC': 'P', 'CCG': 'P', 'CCT': 'P',
'CAC': 'H', 'CAT': 'H', 'CAA': 'Q', 'CAG': 'Q',
'CGA': 'R', 'CGC': 'R', 'CGG': 'R', 'CGT': 'R',
'GTA': 'V', 'GTC': 'V', 'GTG': 'V', 'GTT': 'V',
'GCA': 'A', 'GCC': 'A', 'GCG': 'A', 'GCT': 'A',
'GAC': 'D', 'GAT': 'D', 'GAA': 'E', 'GAG': 'E',
'GGA': 'G', 'GGC': 'G', 'GGG': 'G', 'GGT': 'G',
'TCA': 'S', 'TCC': 'S', 'TCG': 'S', 'TCT': 'S',
'TTC': 'F', 'TTT': 'F', 'TTA': 'L', 'TTG': 'L',
'TAC': 'Y', 'TAT': 'Y', 'TAA': '_', 'TAG': '_',
'TGC': 'C', 'TGT': 'C', 'TGA': '_', 'TGG': 'W'}
```

What you have to do

1. Write a Python programme/script that will take any DNA sequence and translate it into protein using the translation table.
2. What happens if the DNA sequence contains undetermined bases (e.g. N)?
3. Can you generate a translation in all three "forward" frames (transcription is on the top strand, starting at base 1, 2, and 3)?
4. Can you generate a translation in all three "reverse" frames (transcription is on the bottom strand, starting at base end, end-1, and end-2)?

Example input: ATGTTCGGT

Top strand frames

```
ATG TTC GGT --> 1
TGT TCG GT  --> 2
GTT CGG T   --> 3
```

Bottom strand frames

```
      ATG TTC GGT
1 <-- TAC AAG CCA

      AT GTT CGG
2 <-- TA CAA GCC

      A TGT TCG
3 <-- T ACA AGC
```

What to do...(just a suggestion as to what you should ALWAYS do..!)

- outline a workflow/pseudocode on paper/text file
 - figure out what inputs, processes, outputs might (need to) be
 - can any bits be made into a function?
 - test it with some very simple example data to see if it works as you think it should
-
-

Exercises: possible solutions

Interrogating the user...

With the aid of a `function` and a `dictionary`, write an `interactive` Python programme/script that will ask the user the following questions

1. What's your name?
2. How old are you?
3. What is your favourite colour?
4. Do you like Python?
5. The world is flat: True or False?

and then, based on their answers, make some comments back to them (this doesn't have to be at all serious, it's the methods used that we are trying out here....!)

I decided to call my script `ask_the_user.py` and here it is:

```
## Shebang line to indicate what we need to run for the script to work
#!/usr/bin/python3
## Script to ask the user a few questions, with some
## error trapping for inappropriate answers
## v1, 7 Nov 2025, written by s1234546
```

```
## INPUTS
```

```
import os
```

```
# Clear the screen
```

```
os.system('clear')
```

```
# Comments to the screen
```

```
print ("\n\nImported os\n\n")
```

Function to process and check the details that the user provides

Arguments will come in as list from the dictionary of details provided

```
def personal(name,age,col,py,world) :  
    import string  
    print("\nYou have provided the following details:\n\tName: ",name,"\n\tAge: ",age,"\n\tFave colo  
    for character in name:  
        if character not in string.ascii_letters :  
            return print("\nYou are more than just a number, honestly, please start again!")  
    if age > 99 or age < 3 :  
        print("\n"+ name + ", I really dont think your age is credible, do you?!")  
    if col.upper() != "BLACK" :  
        print("\nI really like black, but",col,"is OK too!")  
    else:  
        print("\nI really like black too, excellent choice!")  
    if py.upper()[0] != "Y" :  
        print("\nI am sorry that you dont like Python")  
    else :  
        print("\nGlad you agree that Python is cool...")  
    if world != "False" :  
        return print("\nEither you really DO think the world is flat (it isnt),\nor you havent typed  
    return "OK",print("\nAll OK, thanks a lot.")
```

Process: interact with the user, storing the output in an ordered dictionary

Check the questions are in the right order for the function (or modify function)!!

```
details={}  
details["Name"]    = input("Hi, what is your name? ")  
details["Age"]     = int(input("How old are you? "))  
details["Colour"]  = input("What's your favourite colour? ")  
details["Python"] = input("Do you like Python? ")  
details["World"]   = input("The world is flat: True or False? ")
```

OUTPUTS

This is the line we can use when the dictionary is ordered, as it is in

our server version: Python 3.12.3

... [passing multiple arguments](#)

This does still require the variables to be in the correct order for our function...

```
personal(*list(details.values()))  
  
print("\n\nThis was the dictionary you set up...\n\n",details,"\n\nBye!\n\n")
```

What part of writing the function have I not done?!

Try it!

If I had saved my script as `ask_the_user.py`

```
chmod 700 ask_the_user.py
```

```
./ask_the_user.py
```

Imported os

```
Hi, what is your name? Al
How old are you? 21
What's your favourite colour? black
Do you like Python? yes
The world is flat: True or False? false
```

You have provided the following details:

```
Name: Al
Age: 21
Fave colour: black
Python preference: yes
Flat world: false
```

I really like black too, excellent choice!

Glad you agree that Python is cool...

Either you really DO think the world is flat (it isnt),
or you havent typed False in the correct Python format...

This was the dictionary you set up...

```
{'Name': 'Al', 'Age': 21, 'Colour': 'black', 'Python': 'yes', 'World': 'false'}
```

Bye!

OLDER version of Python: Python 3.5.2

```
Hi, what is your name? Al
How old are you? 99
What's your favourite colour? black
Do you like Python? yes
The world is flat: True or False? True
```

You have provided the following details:

```
Name: 99
Age: yes
Fave colour: Al
Python preference: True
Flat world: black
```

Traceback (most recent call last):

```
File "<stdin>", line 1, in <module>
File "somescript.py", line 39, in <module>
    personal(*list(details.values()))
File "somescript.py", line 15, in personal
    for character in name:
```

TypeError: 'int' object is not iterable

OR, just import the script from within our Python session...

```
import ask_the_user
```

Imported os

```
Hi, what is your name?
```

```
...
```

Do the function variables/values exist still? NO!

details.values()

```
Traceback (most recent call last):  
  File "<stdin>", line 1, in <module>  
NameError: name 'details' is not defined
```

DNA translation

Here's a `dict` that stores a codon usage table for translation:

```
gencode = {  
'ATA': 'I', 'ATC': 'I', 'ATT': 'I', 'ATG': 'M',  
'ACA': 'T', 'ACC': 'T', 'ACG': 'T', 'ACT': 'T',  
'AAC': 'N', 'AAT': 'N', 'AAA': 'K', 'AAG': 'K',  
'AGC': 'S', 'AGT': 'S', 'AGA': 'R', 'AGG': 'R',  
'CTA': 'L', 'CTC': 'L', 'CTG': 'L', 'CTT': 'L',  
'CCA': 'P', 'CCC': 'P', 'CCG': 'P', 'CCT': 'P',  
'CAC': 'H', 'CAT': 'H', 'CAA': 'Q', 'CAG': 'Q',  
'CGA': 'R', 'CGC': 'R', 'CGG': 'R', 'CGT': 'R',  
'GTA': 'V', 'GTC': 'V', 'GTG': 'V', 'GTT': 'V',  
'GCA': 'A', 'GCC': 'A', 'GCG': 'A', 'GCT': 'A',  
'GAC': 'D', 'GAT': 'D', 'GAA': 'E', 'GAG': 'E',  
'GGA': 'G', 'GGC': 'G', 'GGG': 'G', 'GGT': 'G',  
'TCA': 'S', 'TCC': 'S', 'TCG': 'S', 'TCT': 'S',  
'TTC': 'F', 'TTT': 'F', 'TTA': 'L', 'TTG': 'L',  
'TAC': 'Y', 'TAT': 'Y', 'TAA': '_', 'TAG': '_',  
'TGC': 'C', 'TGT': 'C', 'TGA': '_', 'TGG': 'W'}
```

1. Write a Python programme/script that will take any DNA sequence and translate it into protein using the translation table.
2. What happens if the DNA sequence contains undetermined bases (e.g. N)?
3. Can you generate a translation in all three "forward" frames (transcription is on the top strand, starting at base 1, 2, and 3)?
4. Can you generate a translation in all three "reverse" frames (transcription is on the bottom strand, starting at base end, end-1, and end-2)?

Example input: ATGTTCGGT

Top strand frames

```
ATG TTC GGT --> 1  
TGT TCG GT --> 2  
GTT CGG T --> 3
```

Bottom strand frames

```
ATG TTC GGT
```

```
1 <-- TAC AAG CCA  
      AT GTT CGG  
2 <-- TA CAA GCC  
      A TGT TCG  
3 <-- T ACA AGC
```

What to do...(just a suggestion as to what you should ALWAYS do..!)

- outline a workflow/pseudocode on paper/text file
- figure out what inputs, processes, outputs might (need to) be
- can any bits be made into a function?
- test it with some very simple example data to see if it works as you think it should

First, we need to think about splitting up the DNA into codons.

The start position goes up by three each time, and the stop position is always three greater than the start.

So this should suggest that a range might be a useful approach.

Try with some simple known data

```
dna = "ATGTTTCGGT"  
for start in list(range(0,7,3)):  
    codon = dna[start:start+3]  
    print("one codon is " + codon)
```

```
one codon is ATG  
one codon is TTC  
one codon is GGT
```

This works so we need a more generic solution. We always start at zero, and always go up by three, and the middle argument to range (the last start position) has to be two bases back from the end of the sequence.

INPUT

```
dna = "ATGTTTCGTGACGAGGGT"
```

Calculate the start position for the final codon

```
last_codon_start = len(dna) - 2
```

Process the dna sequence in three base chunks

```
for start in list(range(0,last_codon_start,3)):  
    codon = dna[start:start+3]  
    print("one codon is " + codon)
```

```
one codon is ATG
one codon is TTC
one codon is GTG
one codon is ACG
one codon is AGG
```

This version will work for any length DNA sequence, so things are looking good!

Getting the amino acid for a given codon is quite easy, just look it up in the dict

```
gencode = {
'ATA': 'I', 'ATC': 'I', 'ATT': 'I', 'ATG': 'M',
'ACA': 'T', 'ACC': 'T', 'ACG': 'T', 'ACT': 'T',
'AAC': 'N', 'AAT': 'N', 'AAA': 'K', 'AAG': 'K',
'AGC': 'S', 'AGT': 'S', 'AGA': 'R', 'AGG': 'R',
'CTA': 'L', 'CTC': 'L', 'CTG': 'L', 'CTT': 'L',
'CCA': 'P', 'CCC': 'P', 'CCG': 'P', 'CCT': 'P',
'CAC': 'H', 'CAT': 'H', 'CAA': 'Q', 'CAG': 'Q',
'CGA': 'R', 'CGC': 'R', 'CGG': 'R', 'CGT': 'R',
'GTA': 'V', 'GTC': 'V', 'GTG': 'V', 'GTT': 'V',
'GCA': 'A', 'GCC': 'A', 'GCG': 'A', 'GCT': 'A',
'GAC': 'D', 'GAT': 'D', 'GAA': 'E', 'GAG': 'E',
'GGA': 'G', 'GGC': 'G', 'GGG': 'G', 'GGT': 'G',
'TCA': 'S', 'TCC': 'S', 'TCG': 'S', 'TCT': 'S',
'TTC': 'F', 'TTT': 'F', 'TTA': 'L', 'TTG': 'L',
'TAC': 'Y', 'TAT': 'Y', 'TAA': '_', 'TAG': '_',
'TGC': 'C', 'TGT': 'C', 'TGA': '_', 'TGG': 'W'}

dna = "ATGTTTCGTGACGAGGGT"

last_codon_start = len(dna) - 2

for start in list(range(0, last_codon_start, 3)):
    codon = dna[start:start+3]
    aa = gencode[codon]
    print(codon, aa)
```

```
ATG M
TTC F
GTG V
ACG T
AGG R
```

Now all we need to do is build up the protein sequence one amino acid at a time

```
dna = "ATGTTTCGTGACGAGGGT"
last_codon_start = len(dna) - 2
protein = ""
for start in list(range(0, last_codon_start, 3)):
    codon = dna[start:start+3]
    aa = gencode[codon]
    protein = protein + aa
    print(codon, aa, protein)
```

```
ATG M M
TTC F MF
GTG V MFV
ACG T MFVT
AGG R MFVTR
```

We can see how one amino acid gets added to the protein string each time round the loop. At the end of the loop we have the complete translation to protein.

Output the protein when we have finished

```
dna = "ATGTTTCGTGACGAGGGT"
last_codon_start = len(dna) - 2
protein = ""
for start in range(0,last_codon_start,3):
    codon = dna[start:start+3]
    aa = gencode[codon]
    protein = protein + aa
```

What did it generate?

```
print(protein)
```

MFVTR

This is a good candidate for a function, so let's turn it into one!

Think of how to deal with poor inputs e.g. capitalisation issues, # ambiguous bases with no amino acid, etc.

```
def translate_dna(dna):
    last_codon_start = len(dna) - 2
    protein = ""
    for start in list(range(0,last_codon_start,3)):
        codon = dna[start:start+3].upper()
        aa = gencode.get(codon,"X")
        protein = protein + aa
    return protein
```

Test function with some assertions

```
assert translate_dna("ATGTTTCGGT") == "MFG"
assert translate_dna("ATCGATCGATCGTTGCTTATCGATCAG") == "IDRSLLIDQ"
assert translate_dna("actgatcgtagc tag ctgacgtatcgat") == "TDRS_LTYR"
assert translate_dna("ACGATCGATCGT NAC GTACGATCGTACTCG") == "TIDR X VRSYS"
```

No output, so all must be True

Translating the different frames, top or bottom strand

top strand frames, all that is changing is the start position, need to just add it to the function arguments

bottom strand frames, input dna needs to be reverse complemented, then can be processed in exactly the same way as the top strand ones

```
def translate_dna_2(dna,frame=1):
    dna=dna.upper()
    if frame not in [-3,-2,-1,1,2,3]:
        print("Not a valid translation frame,\nHas to be one of these: -3, -2, -1, 1, 2, 3.")
```

```
        return
    if frame in [-3,-2,-1]:
        print("Need to make the reverse complement to do the reverse strand translations...")
        c_dna = dna.replace("G","c").replace("A","t").replace("T","a").replace("C","g").upper()
```

no real builtin reverse method, `reversed()` produces a
different kind of object that we don't really want to deal with!

the following method is a slice `[::-1]`,
is very fast, also regenerates as a string, which is what we want!

```
rc_dna = c_dna[::-1]
print("Was\n",dna,"\nnow\n",rc_dna)
dna = rc_dna
framestart = abs(frame) - 1
last_codon_start = len(dna) - ( 2 )
protein = ""
for start in list(range(framestart,last_codon_start,3)):
    codon = dna[start:start+3]
    aa = gencode.get(codon,"X")
    protein = protein + aa
return protein
```

Check whether it works with/without supplied frame

```
translate_dna_2("ATGTTTCGGT",1)
```

```
'MFG'
```

```
translate_dna_2("ATGTTTCGGT")
```

```
'MFG'
```

Check whether it works with the wrong top strand frame number

```
translate_dna_2("ATGTTTCGGT",4)
```

```
Not a valid translation frame,
Has to be one of these: -3, -2, -1, 1, 2, 3.
Exiting....
```

Check whether it works with the reverse strand frames

```
translate_dna_2("ATGTTTCGGT",-1)
```

```
Need to make the reverse complement to do the reverse strand translations....
Was
  ATGTTTCGGT
now
  ACCGAACAT
'TEH'
```

Check whether it works with the wrong bottom strand frame number

```
translate_dna_2("ATGTTTCGGT",-4)
```

```
Not a valid translation frame,  
Has to be one of these: -3, -2, -1, 1, 2, 3.  
Exiting....
```

The final implementation would remove the internal debugging print statements, perhaps combine a couple of the lines into single lines, and add some more comment lines in the relevant places.

As for all functions, the only things to go in would be the arguments (in this case, a DNA sequence and an optional translation frame), and only output would be returned, in this case it would be our translated sequence in the frame of our choice.

🏠git and GitHub time!

Local repository actions (the absolute minimum set of commands required here...)

git st

On branch master

Changes not staged for commit:

```
(use "git add/rm <file>..." to update what will be committed)
(use "git restore <file>..." to discard changes in working directory)
    deleted:    ../Lecture04/CodeFiles.tar.gz
    modified:   ../Lecture06/run_this_script_v1
```

Untracked files:

```
(use "git add <file>..." to include in what will be committed)
    ../BN.out
    ../BN.sh
    ../Lecture04/Als_ICA/
    ../Lecture04/all_the_things_I_did
    ../Lecture04/outs/
    ../Lecture05/Country.Mozambique.details
    ../Lecture07/Cosmoscarta.nuc.fa
    ../Lecture07/Cosmoscarta.nuc.gis
    ../Lecture07/myfileofaccessions.txt
    ../Lecture12/AJ223353.fasta
    ../Lecture12/AJ223353.genbank
    ../Lecture12/AJ223353_noheader.fasta
    ../Lecture12/AJ223353_noheader.fasta2
    ../Lecture12/AJ223353_noheader.fasta3
    ../Lecture12/All_exons.fasta
    ../Lecture12/All_noncodings.fasta
    ../Lecture12/local_exon01.fasta
    ../Lecture12/local_exon02.fasta
    ../Lecture12/local_intron01.fasta
    ../Lecture12/my_dna.txt
    ../Lecture12/plain_genomic_seq.txt
    ../Lecture12/quick_blast_check.out
    ../Lecture12/remote_exon01.fasta
    ../Lecture12/remote_noncoding01.fasta
    ../Lecture12/remote_noncoding02.fasta
    ../Lecture12/shutilversion.txt
    ../Lecture13/100_199/
```

```
../Lecture13/200_299/  
../Lecture13/300_399/  
../Lecture13/400_499/  
../Lecture13/500_599/  
../Lecture13/600_699/  
../Lecture13/700_799/  
../Lecture13/800_899/  
../Lecture13/900_999/  
../Lecture13/Cleaned_sequences.txt  
../Lecture13/Cleaned_sequences2.txt  
../Lecture13/Cleaned_sequences3.txt  
../Lecture13/Exercise2_coding_sequence.fasta  
../Lecture13/Lecture13.tar.gz  
../Lecture13/dna_files/  
../Lecture13/exons.txt  
../Lecture13/genomic_dna2.txt  
../Lecture13/input.txt  
../Lecture13/remote_exon01.fasta  
../Lecture14/__pycache__/  
../Lecture14/data.csv  
./
```

no changes added to commit (use "git add" and/or "git commit -a")

ls -l

```
ask_the_user.py  
dna_translator2.py  
pickled_all_counts  
__pycache__
```

You might want to add other files?

git add *.py pickled_all_counts

git commit -m "Scripts from the python dictionaries lecture"

```
[master 1d2319e] Scripts from the python dictionaries lecture  
3 files changed, 99 insertions(+)  
create mode 100644 Lecture16/ask_the_user.py  
create mode 100644 Lecture16/dna_translator2.py  
create mode 100644 Lecture16/pickled_all_counts
```

Push our local repository to GitHub

git remote

```
AllLectures  
Lecture07  
origin
```

git push AllLectures master

```
Enumerating objects: 7, done.  
Counting objects: 100% (7/7), done.  
Delta compression using up to 256 threads  
Compressing objects: 100% (6/6), done.  
Writing objects: 100% (6/6), 2.32 KiB | 1.16 MiB/s, done.  
Total 6 (delta 1), reused 0 (delta 0), pack-reused 0  
remote: Resolving deltas: 100% (1/1), completed with 1 local object.  
To https://github.com/B123456-2121/BPSM_LectureStuff.git  
c49df45..1d2319e master -> master
```

git log

```
1d2319e (HEAD -> master, AllLectures/master) Scripts from the python dictionaries lecture  
c49df45 Scripts from the python functions lecture  
3b8bf41 My scripts from the python conditionals exercises Py04
```

```
8f58f4c Scripts from the python lists and loops lecture 3
e444ef8 Scripts from the second BPSM python lecture
6d6f5e7 My python scripts from the first BPSM python lecture
eae65c6 (Lecture07/master) edirect seqs related stuff added
eea4f5e (Lec_6_branch) Kept bash script output files from Lecture 06
f8e5cc0 Added run_this_script_v1 from BPSM Lecture06
ce2c15a nematode BLASTDB stuff added
19cdd2d Added the example_people_data.tsv file from Lecture03
137c3db Decided to keep blastoutput2.out
08d379b Decided to keep myinputfile.tsv
c9b5b18 First attempt at parsing BLAST data
bbf825c My first awk script, Lecture05
68f27fb There were conflicts with someotherfile.sh, kept all changes
83629cb (Version1.2) Final version of the someotherfile.sh shell script prior to merge
0b5425b (tag: v2.0) Updated contents of tar, coolscript
5d8aeea I am not sure cool script is actually any good
bbd36e7 Added my_cool_script
ea60357 Third version of the someotherfile.sh shell script
b6ae4a7 Updated version of tar
ce230e1 (tag: v1.2) Second version of the someotherfile.sh shell script
a5add7c Added the someotherfile.sh shell script
101b700 Additional motif files, and updated tar file
d21cb3d Adding intial tar file
59d1756 CodeFiles all done and tar zipped
6dd85a8 First file added
```

Have your say...

The "Postbox of Truth": is your chance to give anonymous "in course" feedback/comments. Your comments are completely anonymous and confidential, so say what you think, irrespective of whether the comments are positive or otherwise! You can use this multiple times too: I use it throughout the course to enable you to give context-relevant feedback/make suggestions.

Just click the image!

